AI VIET NAM
@aivietnam.edu.vn

Transformation with dbt

Lê Thanh Bảo Trân **Huỳnh Trần Anh Thư**

Proofreading:

Mở đầu

Trong một hệ thống phân tích dữ liệu, cùng một chỉ số kinh doanh có thể cho ra những kết quả khác nhau tùy thuộc vào cách dữ liệu được xử lý. Chẳng hạn, hai dashboard cùng báo cáo doanh thu tháng trước nhưng chênh lệch gần 15% do sử dụng các quy tắc tính toán khác nhau: một bên vẫn tính các đơn hàng đã bị huỷ, bên còn lại loại bỏ chúng; một bên còn gặp sai lệch do phép JOIN giữa bảng thanh toán và bảng chi tiết sản phẩm làm nhân số dòng dữ liệu.

Những vấn đề này không đơn thuần xuất phát từ sai sót trong câu lệnh SQL, mà phản ánh những hạn chế trong cách tổ chức và quản lý quá trình biến đổi dữ liệu. Khi các câu truy vấn được lưu trữ rời rạc, thiếu kiểm soát phiên bản, kiểm thử và tài liệu hóa, việc theo dõi quan hệ phụ thuộc giữa các bảng, phát hiện lỗi và duy trì tính nhất quán của dữ liệu trở nên khó khăn.

Đây cũng chính là những vấn đề mà các thực hành kỹ thuật phần mềm đã hướng đến giải quyết: tổ chức mã nguồn theo module, quản lý phiên bản, kiểm thử và tài liệu hóa. **dbt (data build tool)** mang những thực hành này vào quy trình xử lý dữ liệu, cho phép xây dựng, quản lý và kiểm thử các phép biến đổi dữ liệu bằng SQL theo một cấu trúc có tổ chức và dễ bảo trì hơn.

Phạm vi và cấu trúc bài viết

Bài viết phân tích cách dbt áp dụng các thực hành kỹ thuật phần mềm vào quy trình biến đổi dữ liệu, bao gồm quản lý phiên bản, mô-đun hóa, kiến trúc phân tầng, kiểm thử, tài liệu hóa và CI/CD. Các nội dung được minh họa thông qua một project e-commerce sử dụng dbt Core trên DuckDB.

Bộ dữ liệu minh họa là dữ liệu giả lập gồm 31 bảng với khoảng 264.000 dòng, bao gồm 7.000 khách hàng, 36.000 đơn hàng và 100.000 dòng chi tiết đơn hàng. Các khung **Trong project** được sử dụng để đối chiếu kiến thức lý thuyết với phần cài đặt tương ứng.

Mục lục

I. Vì sao có dbt **4**

I.1. Từ ETL sang ELT 4

I.2. dbt là gì và không phải là gì 4

I.3. Analytics Engineer: vai trò kết nối kỹ thuật và kinh doanh 5

II. dbt hoạt động thế nào **6**

II.1. Model: đơn vị cơ bản của quá trình biến đổi 6

II.2. Cấu trúc project và các lệnh cơ bản 7

II.3. source(), ref() và DAG 7

II.4. Materialization: cùng một SQL, nhiều cách lưu trữ 9

II.4.1. Incremental và những rủi ro thường gặp 10

II.4.2. Lựa chọn materialization cho `fact_sales` 12

III. Thiết kế dữ liệu **13**

III.1. Kiến trúc phân tầng: staging → intermediate → marts 13

III.1.1. Staging: chuẩn hóa dữ liệu nguồn 14

III.1.2. Intermediate: tích hợp dữ liệu và xử lý logic nghiệp vụ 15

III.1.3. Marts: dữ liệu phục vụ phân tích 17

III.1.4. Quy tắc phụ thuộc giữa các tầng 18

III.1.5. Liên hệ với kiến trúc Medallion 18

III.2. Marts và Star Schema (Kimball) 20

III.3. Grain và lỗi fan-out 21

III.3.1. Vì sao lỗi này khó phát hiện 22

III.3.2. Cách sửa: đưa dữ liệu về cùng grain trước khi join 22

III.3.3. Khi cần giữ grain chi tiết sản phẩm 24

III.3.4. Những cách sửa nhanh nên tránh 24

III.3.5. Thực tế làm như thế nào 26

IV. Dữ liệu đáng tin cậy **27**

IV.1. Data tests: kiểm tra dữ liệu thực tế 27

IV.2. Unit tests: kiểm tra logic của model 28

IV.3. Model contracts: cam kết về cấu trúc đầu ra 29

IV.4. Documentation và lineage 31

IV.5. CI/CD 31

V. Tính năng nâng cao **32**

V.1. Jinja và macros: SQL có biến, vòng lặp và hàm 32
V.2. Snapshots: lưu lịch sử thay đổi (SCD Type 2) 33
V.3. Seeds, packages và exposures 34

I. Vì sao có dbt

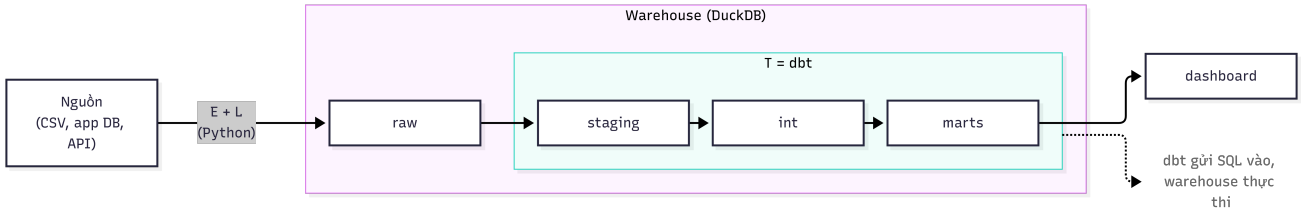
I.1. Từ ETL sang ELT

Trong mô hình ETL (Extract - Transform - Load), dữ liệu được trích xuất từ các nguồn, biến đổi trên một hệ thống xử lý trung gian, sau đó mới được nạp vào Data Warehouse. Cách tiếp cận này phù hợp với các hệ thống truyền thống, khi tài nguyên lưu trữ và năng lực tính toán của Data Warehouse còn hạn chế. Do đó, dữ liệu thường được làm sạch và xử lý trước khi đưa vào kho.

Sự phát triển của các nền tảng Cloud Data Warehouse như Snowflake, BigQuery, Amazon Redshift và các công cụ xử lý dữ liệu như DuckDB đã thúc đẩy sự chuyển dịch sang mô hình ELT (Extract – Load – Transform). Thay vì xử lý dữ liệu trước khi nạp, ELT đưa dữ liệu vào kho trước, sau đó thực hiện các phép biến đổi trực tiếp trên nền tảng lưu trữ bằng SQL.

Cách tiếp cận này cho phép duy trì dữ liệu nguồn ở dạng ban đầu, đồng thời tách biệt quá trình nạp dữ liệu với logic biến đổi. Khi yêu cầu nghiệp vụ thay đổi hoặc phát hiện sai sót trong logic xử lý, các phép biến đổi có thể được thực thi lại từ dữ liệu đã lưu trữ mà không nhất thiết phải trích xuất lại từ nguồn.

Trong kiến trúc này, Python có thể đảm nhiệm việc trích xuất và nạp dữ liệu (E và L), trong khi dbt quản lý các phép biến đổi (T) bằng SQL. dbt không trực tiếp xử lý dữ liệu mà biên dịch các model thành câu lệnh SQL và gửi đến Data Warehouse để thực thi. Hình 1 minh họa luồng ELT được áp dụng trong project.



Hình 1: Luồng ELT trong project: Python đảm nhiệm Extract và Load, dbt quản lý Transform trên DuckDB.

I.2. dbt là gì và không phải là gì

dbt (data build tool) là một công cụ hỗ trợ quản lý quá trình biến đổi dữ liệu trong mô hình ELT. Thay vì trực tiếp trích xuất dữ liệu từ các nguồn hoặc nạp dữ liệu vào kho, dbt tập trung vào việc tổ chức và thực thi các phép biến đổi trên dữ liệu đã được lưu trữ trong Data Warehouse. Các phép biến đổi được định nghĩa chủ yếu bằng SQL, kết hợp với Jinja để hỗ trợ tái sử dụng logic và tạo câu lệnh SQL linh hoạt.

Để hiểu rõ vai trò của dbt, cần phân biệt nó với các thành phần khác trong hệ sinh thái dữ liệu:

dbt LÀ	dbt KHÔNG PHẢI LÀ
Công cụ biên dịch SQL và Jinja thành câu lệnh SQL phù hợp với Data Warehouse	Một hệ quản trị cơ sở dữ liệu hoặc engine trực tiếp xử lý dữ liệu
Công cụ quản lý các model và quan hệ phụ thuộc giữa chúng thông qua DAG	Công cụ trích xuất và nạp dữ liệu (ETL/EL)
Framework hỗ trợ kiểm thử và tài liệu hóa dữ liệu	Công cụ trực quan hóa dữ liệu hoặc xây dựng dashboard
Công cụ quản lý quy trình biến đổi dữ liệu, có thể tích hợp vào pipeline CI/CD	Bộ lập lịch và điều phối workflow đa năng thay thế Airflow hoặc Dagster

Nhận xét

dbt chịu trách nhiệm biên dịch và điều phối các câu lệnh SQL, trong khi việc thực thi và tính toán được giao cho Data Warehouse hoặc database engine tương ứng. Vì vậy, hiệu năng xử lý phụ thuộc chủ yếu vào nền tảng thực thi và cách viết truy vấn SQL. dbt hỗ trợ nhiều nền tảng dữ liệu thông qua các adapter, tuy nhiên khi chuyển đổi giữa các hệ thống, vẫn cần xem xét cấu hình kết nối, khả năng tương thích của adapter và các khác biệt trong cú pháp hoặc hàm SQL đặc thù.

Trong project

Quy trình E và L được triển khai bằng script `scripts/load_csvs.py`. Script đọc 31 file CSV và nạp dữ liệu nguyên trạng vào schema `raw` của DuckDB, mỗi file tương ứng với một bảng (ví dụ: `orders.csv` được nạp thành `raw.orders`). Sau bước này, dbt đảm nhiệm phần T thông qua các model SQL được tổ chức theo từng tầng biến đổi. Toàn bộ dữ liệu được lưu trữ trong file `warehouse/ecommerce.duckdb`.

I.3. Analytics Engineer: vai trò kết nối kỹ thuật và kinh doanh

Khi các nền tảng dữ liệu ngày càng đảm nhiệm nhiều công việc về lưu trữ và tính toán, quá trình biến đổi dữ liệu có thể được thực hiện trực tiếp bằng SQL trong Data Warehouse. Điều này tạo điều kiện hình thành một vai trò chuyên biệt: Analytics Engineer.

Theo cách tiếp cận được dbt Labs phổ biến, Analytics Engineer đóng vai trò kết nối giữa Data Engineer và Data Analyst. Nếu Data Engineer tập trung xây dựng hạ tầng và duy trì các pipeline dữ liệu, còn Data Analyst sử dụng dữ liệu để phân tích và trả lời các câu hỏi kinh doanh, thì Analytics Engineer chịu trách nhiệm biến dữ liệu thô thành các tập dữ liệu có cấu trúc, đáng tin cậy và sẵn sàng cho phân tích.

Điểm khác biệt của vai trò này không chỉ nằm ở việc sử dụng SQL, mà còn ở cách tiếp cận công việc theo quy trình kỹ thuật phần mềm. Thay vì viết các truy vấn riêng lẻ, Analytics Engineer tổ chức logic biến đổi thành các model có thể tái sử dụng, quản lý mã nguồn bằng Git, thực hiện code review, kiểm thử và tích hợp CI. dbt cung cấp một bộ công cụ hỗ trợ trực tiếp cho quy trình làm việc này.

II. dbt hoạt động thế nào

II.1. Model: đơn vị cơ bản của quá trình biến đổi

Trong dbt, model là đơn vị cơ bản dùng để định nghĩa logic biến đổi dữ liệu. Mỗi model thường được viết trong một file `.sql`, chứa một câu truy vấn `SELECT` mô tả dữ liệu đầu ra mong muốn. Thay vì trực tiếp viết các lệnh `CREATE TABLE` hay `INSERT`, người phát triển chỉ cần tập trung vào logic truy vấn; dbt sẽ đảm nhiệm việc biên dịch và tạo các đối tượng tương ứng trong cơ sở dữ liệu.

Ví dụ, model `models/staging/stg_orders.sql` có thể được viết như sau:

models/staging/stg_orders.sql

```
with source as (
    select * from {{ source('shop', 'orders') }}
)
select
    id                        as order_id,
    customer_id,
    cast(order_date as date) as order_date,
    lower(status)             as status,
    subtotal
from source
```

Trong truy vấn trên, `source()` là hàm Jinja giúp tham chiếu đến bảng nguồn đã được khai báo trong project. Khi biên dịch, dbt thay biểu thức này bằng tên bảng đầy đủ tương ứng. Chẳng hạn, câu truy vấn có thể được biên dịch thành:

SQL sau khi biên dịch

```
create or replace view "shop"."dev"."stg_orders" as (
    with source as (
        select * from "shop"."raw"."orders"
    )
    select
        id as order_id,
        customer_id,
        cast(order_date as date) as order_date,
        lower(status) as status,
        subtotal
    from source
);
```

Hai đoạn mã trên thể hiện hai giai đoạn khác nhau: đoạn đầu là SQL do người phát triển viết, còn đoạn sau là SQL đã được dbt biên dịch để thực thi trên DuckDB. Trong ví dụ này, model được cấu hình dưới dạng *view*, do đó dbt tạo hoặc thay thế view `stg_orders` trong schema `dev`. Loại đối tượng được tạo phụ thuộc vào cấu hình materialization của model; ngoài *view*, dbt còn

hỗ trợ *table*, *incremental* và *ephemeral*.

Có thể kiểm tra SQL sau khi biên dịch bằng lệnh `dbt compile`. Lệnh này tạo các file SQL trong thư mục `target/` nhưng không thực thi truy vấn trên DuckDB. Để thực sự tạo hoặc cập nhật các model trong cơ sở dữ liệu, cần sử dụng `dbt run`.

II.2. Cấu trúc project và các lệnh cơ bản

Một project dbt được tổ chức thành các thư mục theo chức năng. Cách tổ chức này giúp tách biệt logic biến đổi, cấu hình, kiểm thử và các thành phần hỗ trợ:

Cấu trúc thư mục một project dbt

```
my_project/
|-- dbt_project.yml       # Cấu hình project
|-- models/
|   |-- staging/          # Model xử lý dữ liệu nguồn
|   |-- intermediate/     # Model xử lý trung gian
|   |-- marts/            # Model phục vụ phân tích
|-- tests/                # Singular tests
|-- macros/               # Các macro Jinja
|-- snapshots/            # Theo dõi thay đổi dữ liệu
|-- seeds/                # Dữ liệu CSV tĩnh
```

Ngoài ra, `profiles.yml` chứa thông tin kết nối đến cơ sở dữ liệu và cấu hình các target như `dev` hoặc `prod`. File này thường được lưu bên ngoài thư mục project.

Trong quá trình phát triển, một số lệnh dbt thường được sử dụng gồm:

Lệnh	Chức năng
`dbt compile`	Biên dịch model thành SQL, không thực thi truy vấn.
`dbt run`	Thực thi và tạo hoặc cập nhật các model.
`dbt test`	Chạy các kiểm thử được khai báo trong project.
`dbt build`	Thực thi các tài nguyên được chọn như models, tests, seeds và snapshots theo thứ tự phụ thuộc.
`dbt docs generate`	Tạo tài liệu mô tả project và thông tin lineage.

Các lệnh này tương ứng với những giai đoạn khác nhau trong vòng đời phát triển dữ liệu: từ biên dịch và thực thi SQL, kiểm tra chất lượng dữ liệu đến tạo tài liệu cho project.

II.3. source(), ref() và DAG

Trong dbt, hai hàm Jinja `source()` và `ref()` được sử dụng để khai báo mối quan hệ giữa các thành phần dữ liệu. Thay vì viết trực tiếp tên bảng trong câu SQL, người phát triển sử dụng hai hàm này để tham chiếu đến nguồn dữ liệu hoặc model khác trong project.

- `source('shop', 'orders')` tham chiếu đến bảng nguồn đã được khai báo trong file YAML. Đây thường là điểm bắt đầu của luồng biến đổi dữ liệu. Việc khai báo source cũng cho phép cấu hình và kiểm tra độ mới của dữ liệu thông qua lệnh `dbt source freshness`.

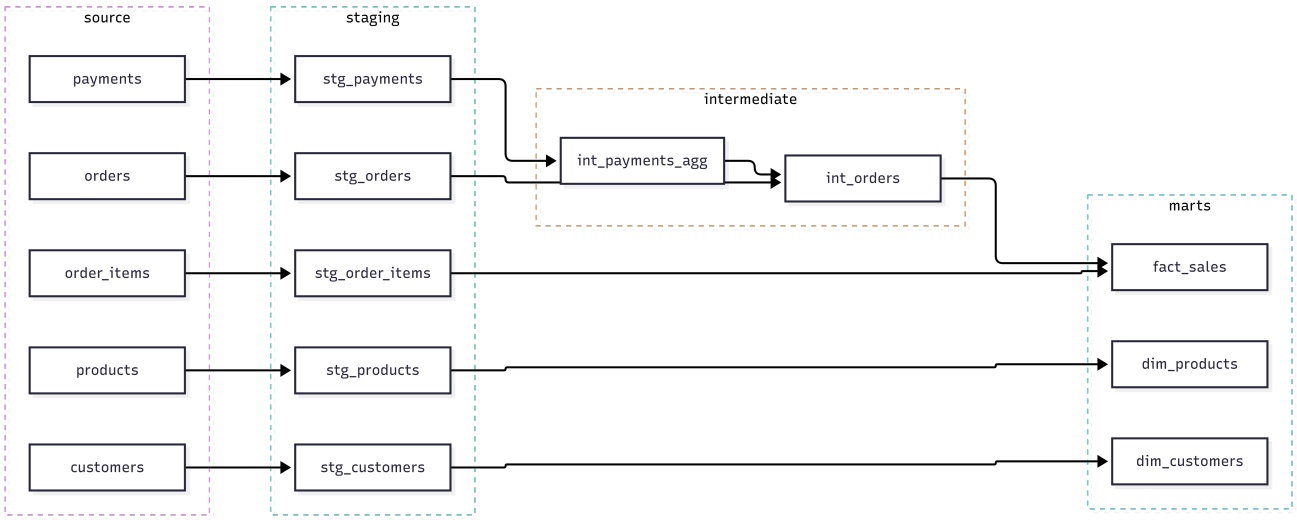
- `ref('stg_orders')` tham chiếu đến một model khác trong cùng project. dbt sử dụng thông tin này để xác định quan hệ phụ thuộc giữa các model.

Ví dụ, các bảng nguồn có thể được khai báo trong file `models/staging/_sources.yml` như sau:

models/staging/_sources.yml

```
sources:
  - name: shop
    schema: raw
    tables:
      - name: orders
      - name: order_items
      - name: payments
```

Khi các model sử dụng `source()` và `ref()`, dbt tự động thu thập thông tin phụ thuộc và xây dựng DAG (Directed Acyclic Graph – đồ thị có hướng không chu trình). Mỗi nút đại diện cho một nguồn dữ liệu hoặc model, còn mỗi cạnh thể hiện mối quan hệ phụ thuộc giữa chúng. Dựa trên đồ thị này, dbt xác định thứ tự thực thi phù hợp và có thể chạy song song các model không phụ thuộc lẫn nhau.



Hình 2: DAG minh họa quan hệ phụ thuộc giữa source và các model trong project.

Một lợi ích quan trọng của `ref()` là khả năng tự động phân giải tên model theo môi trường thực thi. Ví dụ, khi chạy với target `dev`, model `stg_orders` có thể được tạo trong schema `dev`; khi chuyển sang target `prod`, model tương ứng được tạo trong schema `prod`. Nhờ đó, logic SQL không cần thay đổi khi chuyển đổi môi trường. Quy tắc đặt tên schema có thể được tùy chỉnh thông qua macro `generate_schema_name`.

Ý nghĩa trong thực tế

DAG cung cấp thông tin lineage, giúp truy vết nguồn gốc dữ liệu và xác định các model chịu ảnh hưởng khi một thành phần thay đổi. Đồng thời, dbt hỗ trợ chọn một phần DAG để thực thi thay vì phải chạy toàn bộ project. Chẳng hạn, lệnh `dbt build -select +fact_sales` chọn model `fact_sales` cùng các thành phần phụ thuộc phía trước; trong khi `dbt build -select stg_orders+` chọn model `stg_orders` và các model phụ thuộc phía sau nó.

II.4. Materialization: cùng một SQL, nhiều cách lưu trữ

Materialization quyết định cách dbt lưu kết quả của một model trong Data Warehouse. Cùng một câu truy vấn SQL có thể được triển khai thành nhiều loại đối tượng khác nhau, tùy theo nhu cầu sử dụng và đặc điểm dữ liệu.

Loại	Cách hoạt động	Khi nào sử dụng	Trong project
`view`	Lưu định nghĩa truy vấn; dữ liệu được tính khi truy vấn view.	Dữ liệu cần cập nhật theo nguồn, logic biến đổi nhẹ.	staging, intermediate
`table`	Thực thi truy vấn và lưu kết quả thành bảng vật lý; thường được tạo lại khi chạy model.	Dữ liệu được truy vấn thường xuyên, cần tốc độ đọc ổn định.	marts
`incremental`	Lần đầu tạo toàn bộ bảng; các lần sau chỉ xử lý phần dữ liệu được xác định là mới hoặc thay đổi.	Bảng lớn, dữ liệu tăng dần và có cơ chế xác định thay đổi đáng tin cậy.	Chưa sử dụng
`ephemeral`	Không tạo bảng hoặc view riêng; SQL được nhúng vào model phụ thuộc dưới dạng CTE.	Logic trung gian nhỏ, chỉ phục vụ model khác.	Chưa sử dụng

Với các loại materialization như `view` và `table`, có thể thay đổi cách lưu trữ mà không cần viết lại logic SQL. dbt cho phép cấu hình materialization trực tiếp trong model hoặc áp dụng mặc định cho từng thư mục thông qua `dbt_project.yml`:

dbt_project.yml

```
models:
  my_project:
    staging:
      +materialized: view
    intermediate:
      +materialized: view
```

```
    marts:
      +materialized: table
```

Khác với các loại trên, `incremental` yêu cầu người phát triển xác định rõ phần dữ liệu cần xử lý ở những lần chạy tiếp theo. Hai thành phần quan trọng là `is_incremental()` để kiểm tra trạng thái thực thi và `unique_key` để xác định khóa đối chiếu giữa các bản ghi.

II.4.1. Incremental và những rủi ro thường gặp

Ví dụ dưới đây minh họa một model incremental sử dụng `updated_at` làm mốc xác định dữ liệu cần xử lý:

Model incremental với unique_key

```
{{ config(
    materialized='incremental',
    unique_key='order_item_id'
) }}

select *
from {{ ref('int_orders') }}

{% if is_incremental() %}
    where updated_at > (
        select max(updated_at) - interval 3 day
        from {{ this }}
    )
{% endif %}
```

Ở lần chạy đầu tiên, bảng đích chưa tồn tại nên `is_incremental()` trả về `false`; dbt xử lý toàn bộ dữ liệu. Từ những lần chạy sau, điều kiện lọc chỉ lấy các bản ghi có `updated_at` nằm trong khoảng thời gian cần xử lý lại. Trong ví dụ này, khoảng lùi ba ngày được sử dụng để hạn chế bỏ sót dữ liệu đến trễ.

Tuy nhiên, incremental có thể dẫn đến sai lệch dữ liệu nếu điều kiện lọc hoặc cơ chế cập nhật không được thiết kế phù hợp. Hai trường hợp đáng chú ý là:

Trường hợp 1: Dữ liệu đến trễ bị bỏ sót.

Giả sử lần chạy ngày 10 đã xử lý dữ liệu đến hết ngày 10. Sang ngày 11, nguồn phát sinh thêm hai bản ghi:

Thời điểm	Sự kiện	Kết quả
Ngày 10	dbt xử lý dữ liệu đến hết ngày 10.	Mốc `max(updated_at)` là 10/11.
Ngày 11	Đơn #A có `updated_at = 11/11` được nạp vào nguồn.	Bản ghi mới.
Ngày 11	Đơn #B có `updated_at = 09/11` được nạp vào nguồn muộn.	Bản ghi đến trễ.
Ngày 11	dbt lọc dữ liệu theo điều kiện `updated_at > 10/11`.	Lấy #A nhưng bỏ qua #B.
Các lần chạy sau	Mốc thời gian tiếp tục tăng.	#B không được xử lý lại.

Bản ghi #B vẫn tồn tại trong dữ liệu nguồn nhưng không được đưa vào bảng đích. Một cách hạn chế rủi ro là sử dụng khoảng lùi thời gian, chẳng hạn xử lý lại dữ liệu trong ba ngày gần nhất:

Điều kiện lọc với khoảng lùi 3 ngày

```
where updated_at > (
    select max(updated_at) - interval 3 day
    from {{ this }}
)
```

Cách làm này giúp xử lý cả những bản ghi được nạp muộn trong khoảng thời gian đã chọn. Tuy nhiên, nó không đảm bảo phát hiện được các thay đổi xảy ra ngoài khoảng lùi đó.

Trường hợp 2: Bản ghi bị trùng khi không có cơ chế đối chiếu.

Khi một bản ghi đã tồn tại được cập nhật ở nguồn, incremental cần có cơ chế xác định bản ghi tương ứng trong bảng đích. Nếu chỉ thêm dữ liệu mới mà không đối chiếu khóa, cùng một đơn hàng có thể xuất hiện nhiều lần:

Lần chạy	Dữ liệu nguồn	Chỉ thêm bản ghi	Có cơ chế đối chiếu khóa
Ngày 10	#100, paid	#100, paid	#100, paid
Ngày 11	#100, shipped	#100, paid; #100, shipped	#100 được cập nhật thành shipped

Trong ví dụ này, đơn #100 đã chuyển từ trạng thái `paid` sang `shipped`. Nếu bản ghi mới được thêm vào mà không cập nhật bản ghi cũ, đơn hàng sẽ xuất hiện hai lần và có thể khiến doanh thu bị tính trùng.

Khai báo `unique_key` giúp dbt xác định bản ghi cần đối chiếu. Cách xử lý khi khóa đã tồn tại phụ thuộc vào `incremental_strategy` và khả năng hỗ trợ của adapter; do đó cần cấu hình phù hợp để cập nhật hoặc thay thế bản ghi thay vì chỉ thêm dữ liệu.

Hai vấn đề trên có liên quan trực tiếp đến nhau. Khi sử dụng khoảng lùi thời gian, các bản ghi đã được xử lý ở những lần chạy trước cũng có thể được lấy lại. Vì vậy, cần có cơ chế đối chiếu khóa phù hợp để tránh tạo bản ghi trùng lặp.

Nếu cần xây dựng lại toàn bộ bảng, chẳng hạn khi logic SQL thay đổi hoặc dữ liệu đích đã sai lệch, có thể sử dụng:

Build lại toàn bộ bảng

```
dbt build --full-refresh
```

Tùy chọn `-full-refresh` yêu cầu dbt bỏ qua cách xử lý incremental thông thường và xây dựng lại model từ toàn bộ dữ liệu nguồn.

II.4.2. Lựa chọn materialization cho `fact_sales`

Trong project này, model `fact_sales` được triển khai dưới dạng `table` thay vì `incremental`. Quyết định này xuất phát từ đặc điểm của dữ liệu nguồn, cụ thể là bảng `raw.orders` chỉ có cột `created_at` để ghi nhận thời điểm tạo đơn, nhưng không có `updated_at` để xác định thời điểm cập nhật gần nhất.

Khi trạng thái đơn hàng thay đổi, giá trị `order_status_id` được cập nhật nhưng `created_at` không thay đổi. Vì vậy, nếu sử dụng `created_at` làm điều kiện lọc incremental, các thay đổi trạng thái của đơn hàng cũ có thể bị bỏ qua.

Ví dụ, đơn #100 được tạo ngày 01/11 với trạng thái `pending`, sau đó chuyển sang `paid` ngày 03/11 và `shipped` ngày 05/11:

Ngày	Trạng thái trong nguồn	Kết quả lọc incremental	Trong `fact_sales`
01/11	#100 được tạo, trạng thái `pending`.	Lấy bản ghi mới.	#100, pending
03/11	#100 chuyển sang `paid`; `created_at` vẫn là 01/11.	Bỏ qua vì `created_at` không lớn hơn mốc đã xử lý.	#100, pending
05/11	#100 chuyển sang `shipped`.	Tiếp tục bỏ qua.	#100, pending

Như vậy, trạng thái trong bảng đích không phản ánh trạng thái thực tế của đơn hàng. Theo seed `order_status_mapping`, trạng thái `pending` có `counts_as_revenue = false`. Do đó, đơn hàng đã thanh toán và giao thành công vẫn có thể bị loại khỏi phép tính doanh thu.

Sử dụng khoảng lùi thời gian cũng không giải quyết triệt để vấn đề này. Trạng thái đơn hàng có thể thay đổi sau nhiều tuần, chẳng hạn khi giao hàng hoặc hoàn tiền. Nếu không có mốc cập nhật đáng tin cậy, khoảng lùi phải đủ dài để bao phủ toàn bộ lịch sử mới có thể hạn chế việc bỏ sót các thay đổi cũ.

Một hướng khác là sử dụng bảng `raw.audit_logs`, vốn ghi nhận các thao tác `insert`, `update` và `delete` cùng thời điểm `performed_at`. Tuy nhiên, việc kiểm tra dữ liệu cho thấy bảng này chưa đủ tin cậy để làm cơ sở xác định các bản ghi cần cập nhật: trong 421 bản ghi `update` liên quan đến bảng `orders`, chỉ có 37 bản ghi khớp với một đơn hàng thực tế; trong số đó, 20 bản ghi có `performed_at` sớm hơn `created_at` của đơn hàng tương ứng.

Với những giới hạn trên, `table` là lựa chọn phù hợp hơn cho `fact_sales`. Bảng dữ liệu hiện có khoảng 100.000 dòng, và DuckDB có thể xây dựng lại toàn bộ trong vài giây. Việc chạy lại model giúp kết quả phản ánh trạng thái mới nhất của các đơn hàng mà không phụ thuộc vào một cột thời gian cập nhật chưa tồn tại hoặc chưa đáng tin cậy.

Kết luận

Incremental không mặc nhiên tối ưu hơn table. Lựa chọn materialization cần dựa trên quy mô dữ liệu, tần suất cập nhật và khả năng xác định chính xác các bản ghi thay đổi. Trong project này, chi phí build lại còn thấp, trong khi dữ liệu nguồn chưa hỗ trợ incremental một cách đáng tin cậy; vì vậy, sử dụng table là lựa chọn hợp lý.

III. Thiết kế dữ liệu

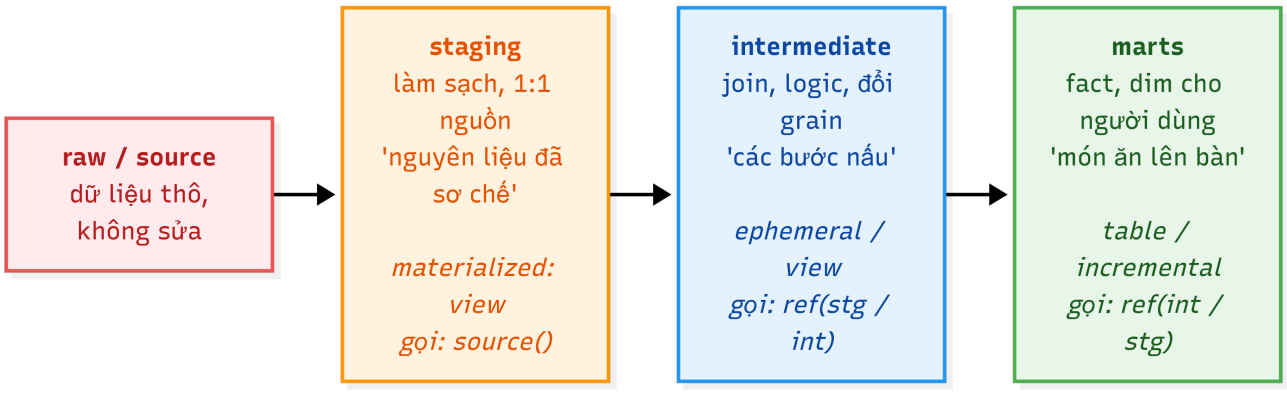
III.1. Kiến trúc phân tầng: staging → intermediate → marts

Tương tự kiến trúc phân tầng trong phát triển phần mềm, project dbt có thể được tổ chức thành nhiều tầng model, trong đó mỗi tầng đảm nhiệm một nhóm nhiệm vụ riêng. Cách tổ chức này giúp tách biệt quá trình làm sạch dữ liệu, xử lý logic nghiệp vụ và xây dựng các tập dữ liệu phục vụ phân tích. Nhờ đó, logic dễ được tái sử dụng, kiểm thử và bảo trì khi dữ liệu nguồn thay đổi.

Kiến trúc thường gặp gồm ba tầng chính:

- **Staging**: tiếp nhận dữ liệu từ source, chuẩn hóa tên cột, kiểu dữ liệu và định dạng giá trị. Thông thường, mỗi model staging tương ứng với một bảng nguồn và sử dụng `view`.
- **Intermediate**: thực hiện các bước biến đổi trung gian như join, tổng hợp, thay đổi grain và xử lý logic nghiệp vụ. Tầng này thường sử dụng `ephemeral` hoặc `view`.
- **Marts**: tổ chức dữ liệu thành các bảng phục vụ trực tiếp cho phân tích và báo cáo, thường bao gồm các mô hình fact và dimension. Tầng này thường sử dụng `table` hoặc `incremental`.

Dữ liệu thường được tổ chức theo chiều từ staging đến intermediate rồi đến marts. Các model ở tầng trước cung cấp dữ liệu cho tầng sau thông qua `ref()`. Việc duy trì chiều phụ thuộc này giúp kiến trúc dễ theo dõi và hạn chế các quan hệ phụ thuộc ngược giữa những tầng.



Hình 3: Kiến trúc phân tầng trong dbt: staging, intermediate và marts.

Theo hướng dẫn tổ chức project của dbt Labs, staging thường được chia theo hệ thống nguồn, trong khi intermediate và marts thường được chia theo lĩnh vực nghiệp vụ. Ví dụ:

Cấu trúc thư mục theo tầng

```
models/
|-- staging/
|   |-- shop/
|   |   |-- _shop_sources.yml
|   |   |-- _shop_models.yml
|   |   |-- stg_orders.sql
|   |   |-- stg_order_items.sql
|   |   |-- stg_payments.sql
|-- intermediate/
|   |-- finance/
|   |   |-- int_payments_agg.sql
|   |   |-- int_orders.sql
|-- marts/
    |-- finance/
    |   |-- fact_sales.sql
    |-- core/
        |-- dim_customers.sql
        |-- dim_products.sql
```

Về quy ước đặt tên, model staging thường sử dụng tiền tố `stg_`, tiếp theo là tên nguồn và thực thể, chẳng hạn `stg_shop_orders`. Với project nhỏ hoặc chỉ có một hệ thống nguồn, có thể rút gọn thành `stg_orders`. Điều quan trọng là duy trì quy ước nhất quán trong toàn bộ project.

III.1.1. Staging: chuẩn hóa dữ liệu nguồn

Staging là tầng đầu tiên trong quá trình biến đổi dữ liệu bằng dbt. Mục tiêu của tầng này là tạo ra phiên bản dữ liệu có cấu trúc thống nhất từ các bảng thô, giúp những model phía sau không phải xử lý lặp lại các khác biệt về tên cột, kiểu dữ liệu và định dạng giá trị.

Theo convention phổ biến, mỗi bảng nguồn được ánh xạ thành một model staging tương ứng. Đây cũng là tầng trực tiếp tham chiếu đến dữ liệu nguồn thông qua hàm `source()`.

Nên thực hiện	Nên tránh
Đổi tên cột để thống nhất quy ước, ví dụ `id` thành `order_id`.	Join nhiều bảng nguồn.
Ép kiểu dữ liệu, ví dụ chuỗi sang `date` hoặc `decimal`.	Tổng hợp bằng `GROUP BY` làm thay đổi grain.
Chuẩn hóa giá trị bằng `lower()`, `trim()` hoặc chuyển đổi đơn vị.	Đưa các quy tắc nghiệp vụ phức tạp như doanh thu thuần hoặc phân loại khách hàng vào đây.
Thực hiện phân loại đơn giản dựa trên một cột nguồn.	Lọc bỏ bản ghi theo các điều kiện nghiệp vụ.

Ví dụ, model `models/staging/shop/stg_payments.sql` thực hiện chuẩn hóa dữ liệu thanh toán:

models/staging/shop/stg_payments.sql

```
with source as (
    select * from {{ source('shop', 'payments') }}
),
renamed as (
    select
        id as payment_id,
        order_id,
        lower(trim(payment_method)) as payment_method,
        amount_cents / 100.0 as amount,
        cast(created_at as timestamp) as paid_at
    from source
)
select * from renamed
```

Trong ví dụ trên, model thực hiện ba nhóm xử lý chính: đổi tên cột, chuẩn hóa giá trị văn bản và chuyển đổi kiểu dữ liệu hoặc đơn vị đo lường. Logic được chia thành các CTE để từng bước biến đổi có thể được đọc và kiểm tra riêng biệt.

Staging thường được materialize dưới dạng `view`, giúp tránh tạo thêm một bản dữ liệu vật lý cho mỗi bảng nguồn. Đối với kiểm thử, có thể khai báo các test cơ bản như `unique` và `not_null` trên khóa chính để phát hiện sớm các vấn đề về chất lượng dữ liệu.

Ý nghĩa

Staging tạo ra một giao diện dữ liệu thống nhất giữa hệ thống nguồn và các model phía sau. Khi cấu trúc nguồn thay đổi, chẳng hạn tên cột hoặc đơn vị đo lường, phần xử lý tương ứng có thể được điều chỉnh tại staging thay vì phải sửa lặp lại ở nhiều model khác nhau.

III.1.2. Intermediate: tích hợp dữ liệu và xử lý logic nghiệp vụ

Intermediate là tầng chứa các bước biến đổi nằm giữa dữ liệu đã chuẩn hóa và các bảng phục vụ phân tích. Tầng này đảm nhiệm những thao tác phức tạp hơn như kết hợp nhiều nguồn, tổng hợp dữ liệu, thay đổi grain và tổ chức logic nghiệp vụ.

Mỗi model intermediate nên đảm nhiệm một nhiệm vụ cụ thể, giúp các bước biến đổi có thể được kiểm tra và tái sử dụng độc lập. Một số nhiệm vụ thường gặp gồm:

- **Thay đổi grain**: tổng hợp dữ liệu thanh toán từ mức một dòng cho mỗi lần thanh toán thành một dòng cho mỗi đơn hàng. Bước này giúp kiểm soát quan hệ giữa các bảng trước khi join và hạn chế hiện tượng fan-out.
- **Tích hợp dữ liệu**: kết hợp thông tin đơn hàng, thanh toán và trạng thái giao hàng thành một tập dữ liệu thống nhất.
- **Chuẩn hóa logic nghiệp vụ**: tập trung các quy tắc như xác định doanh thu thuần hoặc điều kiện ghi nhận doanh thu tại một vị trí có thể tái sử dụng.

- **Tổ chức lại truy vấn**: tách các phép pivot, unpivot hoặc những đoạn SQL phức tạp thành các model riêng để tăng khả năng đọc hiểu và kiểm thử.

Ví dụ, model `models/intermediate/finance/int_orders.sql` kết hợp thông tin đơn hàng với dữ liệu thanh toán đã được tổng hợp:

models/intermediate/finance/int_orders.sql

```
with orders as (
    select * from {{ ref('stg_orders') }}
),
payments as (
    select * from {{ ref('int_payments_agg') }}
)

select
    o.order_id,
    o.customer_id,
    o.order_date,
    o.status,
    coalesce(p.total_paid, 0) as total_paid,
    case
        when o.status in ('cancelled', 'returned') then 0
        else o.subtotal
    end as net_revenue
from orders o
left join payments p using (order_id)
```

Trong model này, `int_payments_agg` đã tổng hợp dữ liệu thanh toán về grain một dòng cho mỗi đơn hàng. Do đó, nếu model này đảm bảo mỗi `order_id` chỉ xuất hiện tối đa một lần, phép join với `stg_orders` sẽ không làm nhân bản các dòng đơn hàng. Đồng thời, quy tắc xác định `net_revenue` được tập trung trong model thay vì lặp lại ở nhiều truy vấn khác nhau.

Về quy ước đặt tên, model intermediate nên thể hiện rõ mục đích hoặc kết quả của bước biến đổi, chẳng hạn `int_payments_agg` hoặc `int_orders_joined_with_payments`. Tầng này thường sử dụng `ephemeral` khi model chỉ phục vụ các model khác và không cần truy vấn độc lập, hoặc `view` khi cần kiểm tra dữ liệu trung gian trực tiếp trong warehouse.

Lưu ý

Không phải project nào cũng cần tạo intermediate cho mọi phép biến đổi. Với những trường hợp đơn giản, chẳng hạn mart chỉ cần kết hợp hai model staging mà không có logic phức tạp hoặc nhu cầu tái sử dụng, có thể thực hiện trực tiếp tại mart. Intermediate nên được bổ sung khi logic cần được tách nhỏ, sử dụng lại ở nhiều model hoặc cần chuẩn hóa grain trước khi kết hợp dữ liệu.

III.1.3. Marts: dữ liệu phục vụ phân tích

Marts là tầng cuối cùng trong kiến trúc phân tầng, cung cấp dữ liệu đã được tổ chức theo nhu cầu phân tích của người dùng, dashboard và các hệ thống khác. Mỗi model thường đại diện cho một thực thể hoặc sự kiện nghiệp vụ, chẳng hạn khách hàng, sản phẩm hoặc giao dịch bán hàng. Dữ liệu tại tầng này thường được tổ chức theo cấu trúc thuận tiện cho truy vấn, có thể kết hợp nhiều thuộc tính liên quan để giảm nhu cầu join ở phía người sử dụng.

Ví dụ, model `models/marts/finance/fact_sales.sql` xây dựng bảng dữ liệu bán hàng ở grain một dòng cho mỗi sản phẩm trong một đơn hàng:

models/marts/finance/fact_sales.sql

```
{{ config(materialized='table') }}

select
    i.order_item_id,  -- grain: 1 dòng / sản phẩm trong đơn
    o.order_id,
    o.customer_id,
    i.product_id,
    o.order_date,
    i.quantity,
    i.price * i.quantity as gross_revenue
from {{ ref('stg_order_items') }} i
join {{ ref('int_orders') }} o using (order_id)
where o.status not in ('cancelled', 'returned')
```

Trong ví dụ này, `fact_sales` kết hợp thông tin chi tiết sản phẩm trong đơn hàng với thông tin đơn hàng tương ứng. Khóa `order_item_id` xác định grain của bảng, do đó một đơn hàng có nhiều sản phẩm sẽ tương ứng với nhiều dòng dữ liệu. Cột `gross_revenue` được tính từ số lượng và đơn giá của từng sản phẩm.

Một số nguyên tắc khi thiết kế marts gồm:

- **Đặt tên**: sử dụng tên thể hiện rõ ý nghĩa nghiệp vụ. Có thể áp dụng quy ước Kimball với tiền tố `fact_` và `dim_`, hoặc sử dụng tên thực thể đơn giản như `orders`, `customers`. Quan trọng là duy trì tính nhất quán trong toàn project.
- **Materialization**: thường sử dụng `table` để lưu kết quả biến đổi, giúp các truy vấn phân tích không phải thực hiện lại toàn bộ logic mỗi lần truy cập. Với bảng dữ liệu lớn và có thể cập nhật từng phần, có thể cân nhắc `incremental`.
- **Chất lượng dữ liệu**: cần chú trọng tài liệu hóa, kiểm thử và khai báo các ràng buộc phù hợp, vì đây là tầng mà người dùng và các hệ thống khác trực tiếp phụ thuộc vào.

Lưu ý

Không nên tạo mart riêng cho từng dashboard, chẳng hạn `orders_for_ceo_dashboard` hoặc `orders_for_marketing_v2`. Cách tổ chức này dễ dẫn đến nhiều phiên bản dữ liệu có định nghĩa nghiệp vụ khác nhau, khiến cùng một chỉ số có thể cho ra kết quả không nhất quán. Thay vào đó, marts nên được thiết kế xoay quanh các thực thể và sự kiện nghiệp vụ có ý nghĩa chung; dashboard sẽ sử dụng các bảng này để lọc, nhóm và trình bày dữ liệu theo nhu cầu.

III.1.4. Quy tắc phụ thuộc giữa các tầng

Để duy trì luồng dữ liệu rõ ràng, các tầng trong kiến trúc dbt nên tuân theo những quy tắc phụ thuộc nhất quán:

Model ở tầng	Được phép tham chiếu	Không nên tham chiếu
staging	`source()`	`ref()` tới các model khác
intermediate	`ref()` tới staging và intermediate khác	`source()`; `ref()` tới marts
marts	`ref()` tới staging, intermediate và (hạn chế) marts khác	`source()`

Các quy tắc trên thể hiện nguyên tắc phụ thuộc một chiều: staging tiếp nhận và chuẩn hóa dữ liệu nguồn, intermediate xử lý logic và tích hợp dữ liệu, còn marts cung cấp kết quả phục vụ phân tích. Việc tham chiếu trực tiếp từ marts đến `source()` sẽ bỏ qua bước chuẩn hóa tại staging, khiến logic xử lý dữ liệu nguồn bị phân tán và khó bảo trì khi cấu trúc nguồn thay đổi.

Đây là quy ước tổ chức kiến trúc, không phải giới hạn mặc định mà dbt tự động áp đặt. Có thể sử dụng package `dbt_project_evaluator` để kiểm tra một số quy tắc tổ chức và phụ thuộc giữa các model, qua đó phát hiện những trường hợp không tuân thủ convention đã chọn.

III.1.5. Liên hệ với kiến trúc Medallion

Trong kiến trúc Lakehouse, Medallion là mô hình tổ chức dữ liệu thành ba tầng Bronze, Silver và Gold. Mô hình này có ý tưởng tương đồng với kiến trúc phân tầng trong dbt, dù hai cách tiếp cận không hoàn toàn tương đương:

Medallion	dbt	Ý nghĩa chung
Bronze	raw / source	Dữ liệu thô được nạp từ hệ thống nguồn, giữ lại cấu trúc ban đầu.
Silver	staging và một phần intermediate	Dữ liệu được làm sạch, chuẩn hóa và tích hợp.
Gold	marts	Dữ liệu được tổ chức phục vụ phân tích và báo cáo.

Điểm khác biệt là Medallion mô tả các lớp dữ liệu trong kiến trúc Lakehouse, còn staging, intermediate và marts là cách tổ chức các model biến đổi trong dbt. Vì vậy, việc đối chiếu trên chỉ mang tính tương đồng về mục đích xử lý dữ liệu, không phải ánh xạ một-một giữa hai mô hình.

Trong project

Project hiện có 54 model, gồm 31 staging (mỗi bảng CSV tương ứng với một model), 5 intermediate và 18 marts. Trong số các marts có 8 dimension, 5 fact thuộc nhóm `core/` và 5 model phục vụ các nhóm nghiệp vụ `sales/`, `customer/` và `operations/`.

Ví dụ, `int_orders_enriched` được tổ chức tại tầng intermediate vì model này đảm nhiệm logic xử lý đơn hàng dùng chung cho nhiều model phía sau. Cụ thể, nó được sử dụng bởi `fact_orders`, gián tiếp phục vụ `fact_sales` thông qua `int_order_items_enriched`, và được sử dụng bởi `dim_customer` thông qua `int_customers_order_history`. Việc tập trung logic xác định đơn hàng đủ điều kiện ghi nhận doanh thu tại đây giúp các model phụ thuộc sử dụng cùng một định nghĩa nghiệp vụ.

Cấu trúc thư mục của project được tổ chức như sau:

Cấu trúc thư mục của project

```
models/
|-- staging/ecommerce/
|   |-- _ecommerce_sources.yml
|   |-- _ecommerce_models.yml
|   |-- stg_ecommerce_orders.sql
|   |-- stg_ecommerce_order_items.sql
|   |-- ...
|
|-- intermediate/
|   |-- int_orders_enriched.sql
|   |-- int_order_items_enriched.sql
|   |-- int_orders_payments_agg.sql
|   |-- int_customers_order_history.sql
|   |-- int_addresses_geography.sql
|
|-- marts/
    |-- core/
    |   |-- dim_date.sql
    |   |-- dim_customer.sql
    |   |-- dim_product.sql
    |   |-- dim_geography.sql
    |   |-- ...
    |   |-- fact_sales.sql
    |   |-- fact_orders.sql
    |   |-- fact_payments.sql
    |   |-- fact_shipments.sql
    |
    |-- sales/
    |   |-- mart_sales_daily.sql
```

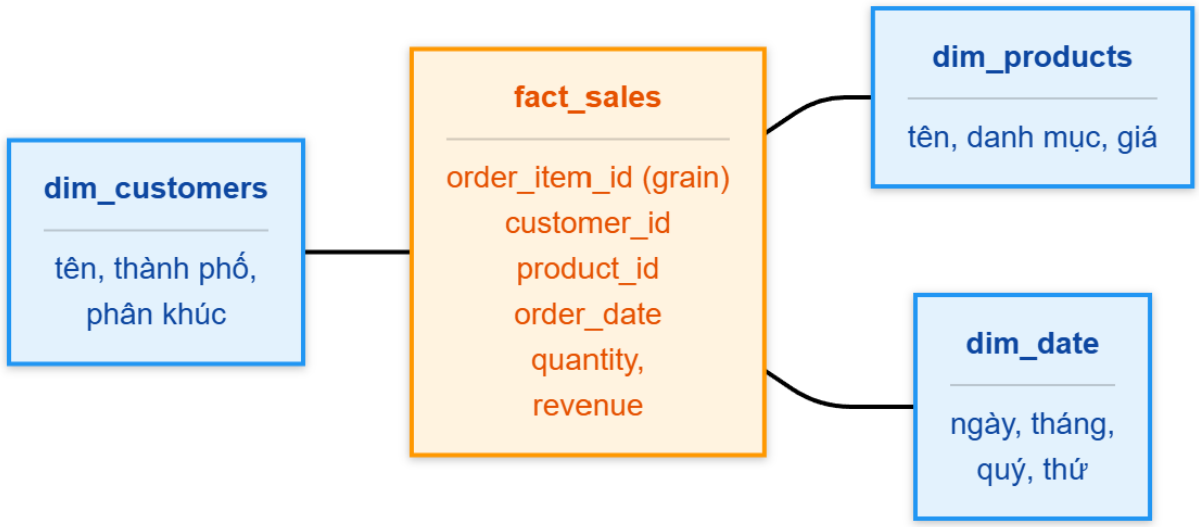
```
|   |-- mart_product_performance.sql
|
|-- customer/
|   |-- mart_customer_360.sql
|
|-- operations/
    |-- fact_inventory_current.sql
    |-- fact_support_tickets.sql
```

III.2. Marts và Star Schema (Kimball)

Một trong những cách thiết kế phổ biến cho tầng marts là Star Schema, được phát triển và phổ biến bởi Ralph Kimball. Mô hình này tổ chức dữ liệu xoay quanh hai thành phần chính:

- **Fact table**: ghi nhận các sự kiện nghiệp vụ có thể đo lường, chẳng hạn bán hàng, thanh toán hoặc giao hàng. Mỗi dòng đại diện cho một sự kiện ở grain xác định, chứa các chỉ số như số lượng, doanh thu và các khóa ngoại liên kết đến dimension.
- **Dimension table**: cung cấp ngữ cảnh mô tả cho các sự kiện, chẳng hạn khách hàng nào mua, sản phẩm nào được bán và giao dịch diễn ra khi nào. Các thuộc tính trong dimension thường được sử dụng để lọc, phân nhóm và phân tích dữ liệu.

Trong Star Schema, fact table nằm ở trung tâm và liên kết trực tiếp với các dimension xung quanh. Cấu trúc này giúp đơn giản hóa truy vấn phân tích, đồng thời hỗ trợ tổng hợp dữ liệu theo nhiều chiều nghiệp vụ khác nhau.



Hình 4: Star Schema: fact table ở trung tâm và các dimension xung quanh.

Trong project

Model `fact_sales` được thiết kế ở grain một dòng cho mỗi sản phẩm trong một đơn hàng. Bảng này liên kết với 7 dimension thông qua surrogate key, bao gồm:

- `dim_date`: thông tin ngày tháng.
- `dim_customer`: thông tin khách hàng.
- `dim_product`: thông tin sản phẩm.
- `dim_geography`: địa điểm giao hàng.
- `dim_promotion`: thông tin chương trình khuyến mãi.
- `dim_shipping_method`: phương thức vận chuyển.
- `dim_order_status`: trạng thái đơn hàng.

Bốn fact table còn lại gồm `fact_orders`, `fact_payments`, `fact_shipments` và `fact_returns`. Mỗi bảng duy trì grain riêng tương ứng với sự kiện nghiệp vụ mà nó ghi nhận, nhưng có thể sử dụng chung các dimension phù hợp để hỗ trợ phân tích xuyên suốt nhiều quy trình.

Đối với các đơn hàng không sử dụng mã giảm giá, khóa liên kết đến `dim_promotion` được gán giá trị tương ứng với bản ghi mặc định "No promotion" (`coupon_id = -1`), thay vì để khóa ngoại ở trạng thái `NULL`. Cách tổ chức này giúp duy trì liên kết với dimension và phân biệt rõ trường hợp không áp dụng khuyến mãi với trường hợp thiếu dữ liệu.

III.3. Grain và lỗi fan-out

Grain (mức độ chi tiết) trả lời câu hỏi: "Một dòng trong bảng này đại diện cho điều gì?" Ví dụ, một dòng trong `stg_orders` đại diện cho một đơn hàng; một dòng trong `stg_order_items` đại diện cho một sản phẩm thuộc đơn hàng; còn một dòng trong `stg_payments` đại diện cho một lần thanh toán. Đây là khái niệm quan trọng khi thiết kế mô hình dữ liệu, vì mỗi phép join đều có khả năng làm thay đổi số dòng và mức độ chi tiết của kết quả.

Xét đơn hàng #1001 có 3 sản phẩm với tổng giá trị 300 và được thanh toán thành 2 lần, lần lượt là 200 và 100. Nếu join trực tiếp `payments` với `order_items` theo `order_id`, kết quả sẽ như sau:

Join không an toàn: kết hợp hai bảng cùng có quan hệ nhiều-một với đơn hàng

Join trực tiếp giữa order_items và payments

```
select
    i.order_item_id,
    p.amount
from {{ ref('stg_order_items') }} i
join {{ ref('stg_payments') }} p
```

```
    on i.order_id = p.order_id
```

Với đơn hàng này, phép join tạo ra $3 \times 2 = 6$ dòng thay vì 3 dòng sản phẩm hoặc 2 dòng thanh toán ban đầu:

Item	Payment	Amount
A	P1	200
A	P2	100
B	P1	200
B	P2	100
C	P1	200
C	P2	100
Tổng		**900 (thực tế 300)**

Đây là hiện tượng fan-out: khi kết hợp hai quan hệ một-nhiều thông qua cùng một khóa, các dòng ở mỗi phía bị lặp lại theo số dòng tương ứng ở phía còn lại. Trong ví dụ trên, mỗi lần thanh toán bị lặp lại 3 lần theo số sản phẩm, khiến `SUM(amount)` tăng từ 300 lên 900.

Hiện tượng này cũng ảnh hưởng đến các chỉ số ở phía sản phẩm. Nếu tính `SUM(price * quantity)` trên kết quả join, tổng giá trị sản phẩm sẽ tăng từ 300 lên 600 vì mỗi sản phẩm bị lặp lại 2 lần. Câu SQL vẫn thực thi bình thường, không phát sinh lỗi cú pháp hay lỗi cơ sở dữ liệu, nhưng kết quả phân tích đã không còn phản ánh đúng dữ liệu ban đầu.

III.3.1. Vì sao lỗi này khó phát hiện

Mức độ nhân bản phụ thuộc vào số dòng của hai bảng ở từng khóa join. Với một đơn hàng có m dòng sản phẩm và n lần thanh toán, phép join trực tiếp có thể tạo ra $m \times n$ dòng. Khi đó, các chỉ số cộng được ở phía thanh toán có thể bị nhân lên m lần, còn các chỉ số ở phía sản phẩm có thể bị nhân lên n lần.

Trong dữ liệu thực tế, nếu phần lớn đơn hàng chỉ có một lần thanh toán, kết quả join của những đơn này vẫn giữ nguyên số dòng như ban đầu. Sai lệch chỉ xuất hiện ở các đơn có nhiều lần thanh toán, chẳng hạn khi khách hàng chia tiền qua nhiều phương thức hoặc thanh toán thành nhiều đợt. Vì vậy, tổng số liệu có thể chỉ lệch một phần nhỏ, khiến lỗi khó được nhận ra khi quan sát dashboard nhưng lại gây sai khác khi đối chiếu với hệ thống kế toán.

Fan-out còn có thể xuất hiện sau khi hệ thống thay đổi nghiệp vụ. Ban đầu, hệ thống thanh toán chỉ hỗ trợ một lần thanh toán cho mỗi đơn hàng nên phép join trực tiếp có thể cho kết quả đúng. Khi bổ sung tính năng thanh toán nhiều đợt, model SQL không cần thay đổi nhưng kết quả tổng hợp đã bắt đầu sai. Đây là lý do cần xem xét grain và quan hệ giữa các bảng ngay từ khi thiết kế model.

III.3.2. Cách sửa: đưa dữ liệu về cùng grain trước khi join

Một cách xử lý phổ biến là tổng hợp từng bảng về cùng grain trước khi kết hợp. Trong ví dụ này, grain chung được chọn là một dòng cho mỗi đơn hàng (`order_id`). Vì vậy, cả dữ liệu thanh toán và chi tiết sản phẩm đều được tổng hợp về cấp đơn hàng:

Tổng hợp về grain đơn hàng trước khi join

```
-- models/intermediate/int_payments_agg.sql
select
    order_id,
    sum(amount) as total_paid
from {{ ref('stg_payments') }}
group by order_id
-- Grain: 1 dòng / order

-- models/intermediate/int_order_items_agg.sql
select
    order_id,
    count(*) as item_count,
    sum(price * quantity) as items_total
from {{ ref('stg_order_items') }}
group by order_id
-- Grain: 1 dòng / order
```

Sau đó, hai model tổng hợp được join với `stg_orders`. Vì mỗi model trung gian chỉ có tối đa một dòng cho mỗi `order_id`, phép join không làm nhân bản các dòng đơn hàng, với điều kiện `stg_orders` cũng đảm bảo tính duy nhất của khóa này.

models/marts/fact_orders.sql

```
-- models/marts/fact_orders.sql (rut gon)
select
    o.order_id,
    i.item_count,
    i.items_total,
    coalesce(p.total_paid, 0) as total_paid
from {{ ref('stg_orders') }} o
left join {{ ref('int_order_items_agg') }} i
    on i.order_id = o.order_id
left join {{ ref('int_payments_agg') }} p
    on p.order_id = o.order_id
```

Kết quả đối với đơn hàng #1001 chỉ còn một dòng:

order_id	item_count	items_total	total_paid
1001	3	300	300

Phép `left join` giúp giữ lại các đơn hàng chưa có thanh toán hoặc chưa có chi tiết sản phẩm tương ứng. Trong trường hợp không có bản ghi thanh toán, `total_paid` nhận giá trị `NULL` và được chuyển thành 0 bằng `coalesce()`.

Cần lưu ý rằng việc đưa hai bảng về cùng grain chỉ giải quyết vấn đề nhân bản do fan-out. Các khóa join vẫn phải được kiểm tra tính duy nhất và tính toàn vẹn bằng các test phù hợp, chẳng hạn `unique` và `not_null` trên `order_id` của bảng đơn hàng và các model tổng hợp.

III.3.3. Khi cần giữ grain chi tiết sản phẩm

Không phải lúc nào cũng có thể tổng hợp dữ liệu về cấp đơn hàng. Ví dụ, `fact_sales` cần giữ grain một dòng cho mỗi sản phẩm trong đơn hàng. Nếu join trực tiếp `total_paid` vào từng dòng sản phẩm, giá trị thanh toán cấp đơn sẽ bị lặp lại và không thể cộng trực tiếp trên toàn bảng.

Có hai cách xử lý phổ biến:

- **Giữ chỉ số ở đúng grain**: lưu `total_paid` trong `fact_orders`, nơi mỗi dòng đại diện cho một đơn hàng. Khi cần phân tích thanh toán ở cấp đơn, truy vấn trực tiếp bảng này thay vì cộng lại giá trị bị lặp trong `fact_sales`.
- **Phân bổ giá trị xuống từng dòng**: nếu nghiệp vụ yêu cầu phân tích số tiền thanh toán theo sản phẩm, có thể phân bổ theo tỷ lệ giá trị từng dòng trên tổng giá trị sản phẩm của đơn hàng:

$$\text{allocated_paid} = \text{total_paid} \times \frac{\text{line_total}}{\text{items_total}}$$

  Với điều kiện `items_total` khác 0 và toàn bộ giá trị được phân bổ theo cùng một quy tắc, tổng số tiền sau phân bổ của các dòng trong một đơn sẽ bằng `total_paid`.

Cách phân bổ chỉ phù hợp khi có cơ sở nghiệp vụ cho việc phân bổ đó. Nếu cần phản ánh chính xác từng giao dịch thanh toán, nên duy trì dữ liệu ở grain thanh toán thay vì phân bổ số tiền xuống sản phẩm một cách tùy ý.

III.3.4. Những cách sửa nhanh nên tránh

Khi phát hiện fan-out, một số cách sửa có thể cho ra kết quả đúng trong ví dụ nhỏ nhưng không đảm bảo tính đúng đắn khi dữ liệu thay đổi.

- `sum(distinct p.amount)`: chỉ phù hợp nếu mỗi khoản thanh toán có số tiền khác nhau. Ví dụ, một đơn hàng được thanh toán hai lần, mỗi lần 100, thì tổng đúng là 200 nhưng `sum(distinct p.amount)` chỉ trả về 100.
- `select distinct` sau khi join: `distinct` chỉ loại bỏ các dòng trùng nhau trên toàn bộ những cột được chọn. Vì vậy, kết quả phụ thuộc vào danh sách cột trong câu truy vấn. Với 6 dòng sau khi join của đơn #1001:

Cột được select	Số dòng sau distinct	Tổng amount	Nhận xét
item, payment, amount	6	900	Sai. Các dòng có item khác nhau vẫn được giữ lại, dù cùng một payment.
payment, amount	2	300	Đúng trong ví dụ này, nhưng kết quả chỉ còn thông tin thanh toán; việc join với `order_items` không còn ý nghĩa.
amount	2	300	Đúng trong ví dụ này vì hai khoản thanh toán có số tiền khác nhau. Nếu chúng bằng nhau, `distinct` sẽ gộp thành một dòng.

- Chia ngược lại cho hệ số nhân, chẳng hạn `sum(p.amount) / count(distinct i.order_item_id)`. Ý tưởng là mỗi payment bị lặp lại theo số lượng item của đơn hàng, nên có thể chia tổng tiền cho số item để khôi phục kết quả. Tuy nhiên, cách này không đúng trong mọi trường hợp vì số item có thể khác nhau giữa các đơn hàng.

  Xét dữ liệu của ba đơn hàng:

  `order_items`:

order_id	item	price
1001	A	100
1001	B	100
1001	C	100
1002	D	50

  `payments`:

order_id	payment	amount
1001	P1	200
1001	P2	100
1002	P3	50
1003	P4	80

  Đơn #1001 có 3 item và 2 lần thanh toán; đơn #1002 có 1 item và 1 lần thanh toán. Đơn #1003 đã có thanh toán nhưng chưa có dữ liệu item, chẳng hạn do dữ liệu chi tiết đơn hàng được cập nhật trễ.

  Trường hợp 1: Tính trên nhiều đơn hàng cùng lúc.

Đơn	Số item	Số payment	Số dòng sau join	Tổng amount sau join	Giá trị đú
#1001	3	2	6	900	300
#1002	1	1	1	50	50
Cả hai đơn	4	3	7	950	350

Chia tổng trên cả bảng cho số item sẽ cho kết quả 950/4 = 237,5, thay vì 350. Nguyên nhân là đơn #1001 bị nhân 3 lần, trong khi đơn #1002 chỉ bị nhân 1 lần. Không tồn tại một hệ số chung để chia cho toàn bộ kết quả.

Trường hợp 2: Join thêm một bảng có quan hệ một-nhiều.

Nếu mỗi dòng sau join tiếp tục bị nhân đôi bởi một bảng khác, tổng amount của đơn #1001 có thể tăng từ 900 lên 1.800. Chia cho 3 item vẫn cho kết quả 600, gấp đôi giá trị đúng là 300.

Trường hợp 3: Đơn hàng không có item.

Với đơn #1003, nếu giữ lại payment bằng `left join`, số lượng item là 0. Khi đó, phép chia cho `count(distinct i.order_item_id)` có thể gây lỗi chia cho 0 hoặc trả về `NULL`, tùy hệ quản trị cơ sở dữ liệu.

III.3.5. Thực tế làm như thế nào

Thay vì sửa kết quả sau khi join, cách tiếp cận an toàn là xác định grain của từng model, tổng hợp các bảng về cùng grain rồi mới thực hiện phép join. Đây cũng là cách tổ chức logic thường được áp dụng trong các project dbt:

- Project mẫu `jaffle_shop` của dbt Labs sử dụng các model trung gian để tổ chức và tổng hợp dữ liệu trước khi đưa vào các model phục vụ phân tích.
- Trong kiến trúc phân tầng, các phép join và aggregation phức tạp thường được đặt ở tầng intermediate.
- Mỗi model mart nên ghi rõ grain trong phần `description`, đồng thời bổ sung test `unique` và `not_null` trên các cột khóa xác định grain.
- Với các chỉ số quan trọng như doanh thu, có thể viết thêm singular test để đối chiếu tổng ở mart với kết quả tính toán từ các model đầu vào ở grain phù hợp.
- Một số công cụ BI và semantic layer hỗ trợ khai báo quan hệ, grain hoặc quy tắc tổng hợp để hạn chế fan-out. Tuy nhiên, hiệu quả phụ thuộc vào cách mô hình hóa và cấu hình của từng công cụ.

Điểm nhấn

Trước mỗi phép join, hãy xác định: grain của hai bảng đầu vào là gì và grain của bảng kết quả sẽ là gì? Ghi rõ grain trong phần mô tả model và sử dụng test để kiểm tra các ràng buộc tương ứng.

Trong project

`order_total` là chỉ số ở grain đơn hàng, được lưu trong `fact_orders`. Nếu join `fact_orders` với `fact_sales` (grain dòng sản phẩm) rồi tính `sum(order_total)`, giá trị của mỗi đơn hàng sẽ bị lặp lại theo số dòng sản phẩm thuộc đơn đó. Trong project, tổng tăng từ

10,8 triệu lên 40,2 triệu, tương đương khoảng 3,7 lần, mà truy vấn vẫn chạy bình thường. File `analyses/fanout_revenue_inflation.sql` đặt hai kết quả cạnh nhau để minh họa sai lệch này.

Để hạn chế vấn đề, project giữ các chỉ số ở cấp đơn hàng như phí vận chuyển, thuế và tổng đơn trong `fact_orders`; `fact_sales` chỉ chứa các chỉ số phù hợp với grain dòng sản phẩm. Ngoài ra, dữ liệu thanh toán được tổng hợp về một dòng cho mỗi đơn hàng trong `int_orders_payments_agg` trước khi join với các model ở grain đơn hàng.

IV. Dữ liệu đáng tin cậy

IV.1. Data tests: kiểm tra dữ liệu thực tế

Data test là các phép kiểm tra (assertion) được thực hiện trên dữ liệu của các model sau khi chúng được tạo. Về bản chất, mỗi test là một truy vấn SQL tìm những dòng vi phạm điều kiện đã khai báo. Nếu truy vấn không trả về dòng nào, test được xem là `PASS`; ngược lại, test sẽ báo lỗi hoặc cảnh báo tùy cấu hình.

Generic tests là các test có sẵn của dbt, được khai báo trong YAML. Bốn loại cơ bản gồm:

- `unique`: giá trị trong cột phải duy nhất.
- `not_null`: cột không được chứa giá trị NULL.
- `accepted_values`: giá trị phải thuộc danh sách cho phép.
- `relationships`: giá trị khóa ngoại phải tồn tại trong cột được tham chiếu.

Ví dụ kiểm tra model `fact_sales`:

models/marts/schema.yml

```
# models/marts/schema.yml
models:
  - name: fact_sales
    description: "Mỗi dòng là một sản phẩm trong một đơn hàng."
    columns:
      - name: order_item_id
        data_tests: [unique, not_null]
      - name: status
        data_tests:
          - accepted_values:
              values: ['completed', 'shipped',
                       'cancelled', 'returned']
      - name: customer_id
        data_tests:
```

```
      - relationships:
          to: ref('dim_customer')
          field: customer_id
```

Trong ví dụ, `unique` và `not_null` giúp kiểm tra khóa xác định grain của `fact_sales`. Test `relationships` kiểm tra các khách hàng được tham chiếu đều tồn tại trong `dim_customer`.

Singular tests là các test do người phát triển tự viết bằng SQL trong thư mục `tests/`. Loại test này phù hợp với những quy tắc nghiệp vụ không thể biểu diễn bằng các generic tests có sẵn.

Ví dụ, kiểm tra tổng giá trị các item có khớp với subtotal của đơn hàng hay không:

tests/assert_items_sum_to_subtotal.sql

```
-- tests/assert_items_sum_to_subtotal.sql
-- Trả về các đơn hàng có subtotal bị lệch
select
    o.order_id,
    o.subtotal,
    coalesce(sum(i.price * i.quantity), 0) as items_total
from {{ ref('stg_orders') }} o
left join {{ ref('stg_order_items') }} i
    using (order_id)
group by o.order_id, o.subtotal
having abs(
    o.subtotal - coalesce(sum(i.price * i.quantity), 0)
) > 0.01
```

Test trên sử dụng `left join` để không bỏ qua những đơn hàng chưa có dữ liệu item. Tuy nhiên, quy tắc so sánh này chỉ phù hợp nếu `subtotal` thực sự được định nghĩa bằng tổng `price * quantity`, chưa bao gồm các khoản giảm giá hoặc điều chỉnh khác.

Ngoài ra, dbt cho phép cấu hình mức độ nghiêm trọng của test thông qua `severity`: `error` (mặc định, làm test thất bại) hoặc `warn` (chỉ cảnh báo).

IV.2. Unit tests: kiểm tra logic của model

Nếu data test kiểm tra dữ liệu thực tế có thỏa mãn các điều kiện hay không, thì unit test kiểm tra logic SQL của model có hoạt động đúng với những đầu vào cụ thể hay không.

Tính năng unit test được giới thiệu từ dbt 1.8. Thay vì chạy model trên toàn bộ dữ liệu nguồn, người phát triển khai báo một tập dữ liệu đầu vào giả lập (`given`) và kết quả mong đợi (`expect`). dbt sử dụng dữ liệu giả này để thực thi logic của model, sau đó so sánh kết quả thực tế với kết quả mong đợi.

Ví dụ, kiểm tra logic tính doanh thu của model `int_orders`, trong đó đơn bị hủy không được tính doanh thu:

Unit test cho model int_orders

```
unit_tests:
  - name: revenue_excludes_cancelled_orders
    model: int_orders
    given:
      - input: ref('stg_orders')
        rows:
          - {order_id: 1, status: 'completed', subtotal: 100}
          - {order_id: 2, status: 'cancelled', subtotal: 50}
    expect:
      rows:
        - {order_id: 1, net_revenue: 100}
        - {order_id: 2, net_revenue: 0}
```

Trong ví dụ này, unit test kiểm tra hai trường hợp: đơn hàng hoàn tất được tính doanh thu bình thường và đơn hàng bị hủy có doanh thu bằng 0. Dữ liệu đầu vào được khai báo riêng cho test, không làm thay đổi dữ liệu nguồn thực tế.

	Data test	**Unit test**
Kiểm tra	Dữ liệu thực tế của model	Logic SQL trên dữ liệu giả lập
Phát hiện	Dữ liệu NULL, khóa trùng, khóa ngoại không hợp lệ, vi phạm quy tắc nghiệp vụ	Sai công thức, sai điều kiện lọc, xử lý sai trường hợp biên
Dữ liệu sử dụng	Dữ liệu trong warehouse	Dữ liệu đầu vào do người phát triển khai báo
Thời điểm chạy	Sau khi model được tạo	Trong quá trình phát triển và CI

IV.3. Model contracts: cam kết về cấu trúc đầu ra

Data test kiểm tra các giá trị trong dữ liệu sau khi model được tạo. Trong khi đó, model contract (được giới thiệu từ dbt 1.5) xác định trước cấu trúc đầu ra mà model phải tuân thủ, bao gồm tên cột và kiểu dữ liệu. Khi bật `enforced: true`, dbt kiểm tra cấu trúc model trong quá trình build và báo lỗi nếu kết quả không đáp ứng contract.

Ví dụ khai báo contract cho model `fact_sales`:

Model contract cho fact_sales

```
models:
  - name: fact_sales
    config:
      contract:
        enforced: true
    columns:
      - name: order_item_id
        data_type: integer
```

```
      constraints:
        - type: not_null
    - name: revenue
      data_type: decimal(18,2)
```

Trong ví dụ, model phải có cột `order_item_id` với kiểu `integer` và cột `revenue` với kiểu `decimal(18,2)`. Ràng buộc `not_null` khai báo thêm yêu cầu đối với cột khóa. Khả năng thực thi các constraint phụ thuộc vào adapter và hệ quản trị cơ sở dữ liệu; không nên mặc định mọi constraint đều được hỗ trợ hoặc thực thi giống nhau trên mọi nền tảng.

Nhận xét

Ba cơ chế bảo vệ các khía cạnh khác nhau của model: unit test kiểm tra logic biến đổi, model contract kiểm tra cấu trúc đầu ra, còn data test kiểm tra dữ liệu thực tế.

Trong project

Project hiện có 232 data tests, được thực thi trong quy trình `dbt build`. Khi một model hoặc test thất bại, các node phụ thuộc phía sau có thể bị bỏ qua do không thể tiếp tục thực thi an toàn. Các nhóm kiểm thử chính gồm:

- `unique` và `not_null` trên khóa chính của các source, model staging, dimension và fact.
- `relationships` để kiểm tra khóa ngoại từ fact tới dimension.
- `accepted_values` trên các cột trạng thái.
- 3 singular tests cho các quy tắc nghiệp vụ.

Kết quả lần `dbt build` gần nhất:

Kết quả dbt build

```
[WARNING]: in test warn_orders_without_items
[WARNING]: Got 2258 results, configured to warn if > 0
Done. PASS=287 WARN=1 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=288
```

Lần chạy này không có test nào thất bại. Cảnh báo duy nhất đến từ test `warn_orders_without_items`: có 2.258 đơn hàng không có sản phẩm đi kèm. Đây là trường hợp được ghi nhận có chủ đích trong bộ dữ liệu giả lập, khi một số đơn chỉ có phí vận chuyển mà chưa có dòng sản phẩm. Test được cấu hình với `severity: warn`, nên vẫn hiển thị cảnh báo nhưng không làm pipeline thất bại.

Lưu ý rằng `TOTAL=288` là tổng số node được ghi nhận trong lần build, không phải số lượng data tests. Hai số liệu 232 và 288 phản ánh hai phạm vi thống kê khác nhau.

IV.4. Documentation và lineage

Trong dbt, tài liệu không cần được viết tách biệt khỏi mã nguồn. Các thông tin như `description` của model, cột và nguồn dữ liệu được khai báo trực tiếp trong YAML. Kết hợp với DAG mà dbt xây dựng từ `source()` và `ref()`, những thông tin này được tổng hợp thành website tài liệu thông qua hai lệnh:

- `dbt docs generate`: tạo bộ tài liệu tĩnh, bao gồm metadata, mô tả model, thông tin kiểm thử và lineage.
- `dbt docs serve`: khởi chạy web server để xem tài liệu trên trình duyệt.

Một thành phần quan trọng của dbt docs là lineage graph. Công cụ này cho phép truy vết dữ liệu theo hai chiều: từ nguồn đến model đầu ra (upstream) và từ một model đến các thành phần sử dụng nó (downstream).

Nhận xét

Lineage giúp trả lời câu hỏi về nguồn gốc của một chỉ số, chẳng hạn doanh thu được tính từ những bảng nào và trải qua các bước biến đổi nào. Trong project, lineage của `fact_sales` cho thấy dữ liệu đi qua các model `int_order_items_enriched` và `int_orders_enriched`, tiếp tục truy ngược về các model staging và các bảng nguồn trong schema `raw`.

IV.5. CI/CD

Vì model, cấu hình và test đều được quản lý dưới dạng file trong Git, dbt có thể tích hợp vào quy trình CI/CD. Khi một Pull Request được tạo, hệ thống CI có thể tự động cài đặt dependencies, kiểm tra cú pháp và chạy các model cùng test trên một schema riêng. Nhờ đó, thay đổi được kiểm tra trước khi merge vào nhánh chính hoặc triển khai lên production, hạn chế ảnh hưởng đến dữ liệu đang được sử dụng.

Với project lớn, việc build lại toàn bộ model trong mỗi Pull Request có thể tốn thời gian. dbt hỗ trợ Slim CI, cho phép chỉ chọn các model đã thay đổi cùng những model downstream phụ thuộc vào chúng:

Slim CI trong Pull Request

```
dbt build --select state:modified+ --defer --state ./prod-artifacts
```

Trong đó:

- `state:modified+`: chọn các model có thay đổi so với trạng thái được lưu trong thư mục state, cùng các model downstream của chúng.
- `-defer`: với những model không được build trong lần chạy CI, dbt có thể tham chiếu đến relation tương ứng trong môi trường production thay vì yêu cầu tạo lại chúng.
- `-state ./prod-artifacts`: chỉ định thư mục chứa các artifact của lần build trước, thường bao gồm `manifest.json`, để dbt so sánh trạng thái và xác định các model cần build.

Cách tiếp cận này giúp giảm số lượng model phải thực thi trong CI, đồng thời vẫn kiểm tra được ảnh hưởng của thay đổi lên các model phụ thuộc. Để Slim CI hoạt động đúng, cần lưu trữ artifact của môi trường production và cấu hình schema riêng cho lần chạy CI.

V. Tính năng nâng cao

V.1. Jinja và macros: SQL có biến, vòng lặp và hàm

Jinja là một template engine mã nguồn mở được viết bằng Python. Trong dbt, Jinja cho phép bổ sung biến, điều kiện, vòng lặp và các hàm tái sử dụng vào câu SQL. Warehouse không trực tiếp xử lý Jinja mà chỉ nhận câu SQL sau khi dbt render và compile.

Jinja có ba nhóm cú pháp cơ bản:

Cú pháp	Chức năng	Ví dụ
`{{ }}`	Biểu thức: tính giá trị và chèn kết quả vào nội dung được render	`{{ ref('stg_orders') }}`
`{% %}`	Câu lệnh: điều kiện, vòng lặp, gán biến, định nghĩa macro	`{% if is_incremental() %}`
`{# #}`	Chú thích Jinja, bị loại bỏ khi render	`{# giải thích macro #}`

Macro và cách sử dụng trong model.

Macro là các hàm template được định nghĩa bằng Jinja, thường đặt trong thư mục `macros/`. Chúng cho phép đóng gói những đoạn logic được sử dụng lặp lại thành một thành phần có thể gọi từ nhiều model, giúp giảm trùng lặp và duy trì tính nhất quán theo nguyên tắc DRY (Don't Repeat Yourself).

Ví dụ, định nghĩa macro chuyển một giá trị tiền tệ sang VND:

macros/to_vnd.sql

```
-- macros/to_vnd.sql
{% macro to_vnd(col, rate=25000) %}
round({{ col }} * {{ rate }}, 0)
{% endmacro %}

-- Trong model
select
    order_id,
    {{ to_vnd('subtotal') }} as subtotal_vnd
from {{ ref('stg_orders') }}
```

Khi compile, lời gọi macro được render thành một biểu thức SQL tương ứng:

Kết quả sau khi render

```
round(subtotal * 25000, 0) as subtotal_vnd
```

Ngoài macro, Jinja còn hỗ trợ điều kiện và vòng lặp. Chẳng hạn, `is_incremental()` cho phép thêm điều kiện lọc chỉ khi model đang chạy ở chế độ incremental:

Điều kiện lọc khi chạy incremental

```
select *
from {{ ref('stg_orders') }}

{% if is_incremental() %}
where created_at >= (
    select coalesce(max(created_at), '1900-01-01')
    from {{ this }}
)
{% endif %}
```

Ở đây, phần điều kiện `where` chỉ được đưa vào SQL đã render khi model chạy incremental. Nhờ đó, cùng một file SQL có thể tạo ra câu truy vấn khác nhau tùy ngữ cảnh thực thi.

Trong project

Project có ba macro, mỗi macro đảm nhiệm một tác vụ cụ thể:

- `to_money(col)`: chuẩn hóa các cột tiền tệ về kiểu `decimal(18,2)` trong các model staging.
- `date_key(col)`: chuyển giá trị ngày thành khóa ngày dạng số nguyên để sử dụng trong các model phân tích.
- `generate_schema_name`: tùy chỉnh tên schema được tạo theo cấu hình của dbt, phục vụ việc tổ chức model trong các môi trường khác nhau.

Các macro này giúp tập trung những quy tắc dùng chung vào một nơi. Khi cần thay đổi cách chuẩn hóa kiểu tiền hoặc tạo khóa ngày, chỉ cần cập nhật macro thay vì sửa từng model.

V.2. Snapshots: lưu lịch sử thay đổi (SCD Type 2)

Nhiều bảng nguồn chỉ lưu trạng thái hiện tại của một bản ghi. Khi dữ liệu thay đổi, giá trị cũ bị ghi đè nên khó truy lại trạng thái tại một thời điểm trong quá khứ. Snapshot trong dbt giải quyết vấn đề này bằng cách lưu lại các phiên bản của bản ghi qua nhiều lần kiểm tra dữ liệu nguồn.

Cơ chế này tương ứng với Slowly Changing Dimension Type 2 (SCD Type 2). Khi phát hiện một bản ghi thay đổi, dbt đánh dấu phiên bản cũ đã kết thúc và tạo một phiên bản mới. Snapshot thường bổ sung các cột metadata như:

- `dbt_valid_from`: thời điểm phiên bản bắt đầu có hiệu lực.
- `dbt_valid_to`: thời điểm phiên bản kết thúc; giá trị NULL thường biểu thị phiên bản hiện tại.

- `dbt_scd_id`: định danh của phiên bản snapshot.

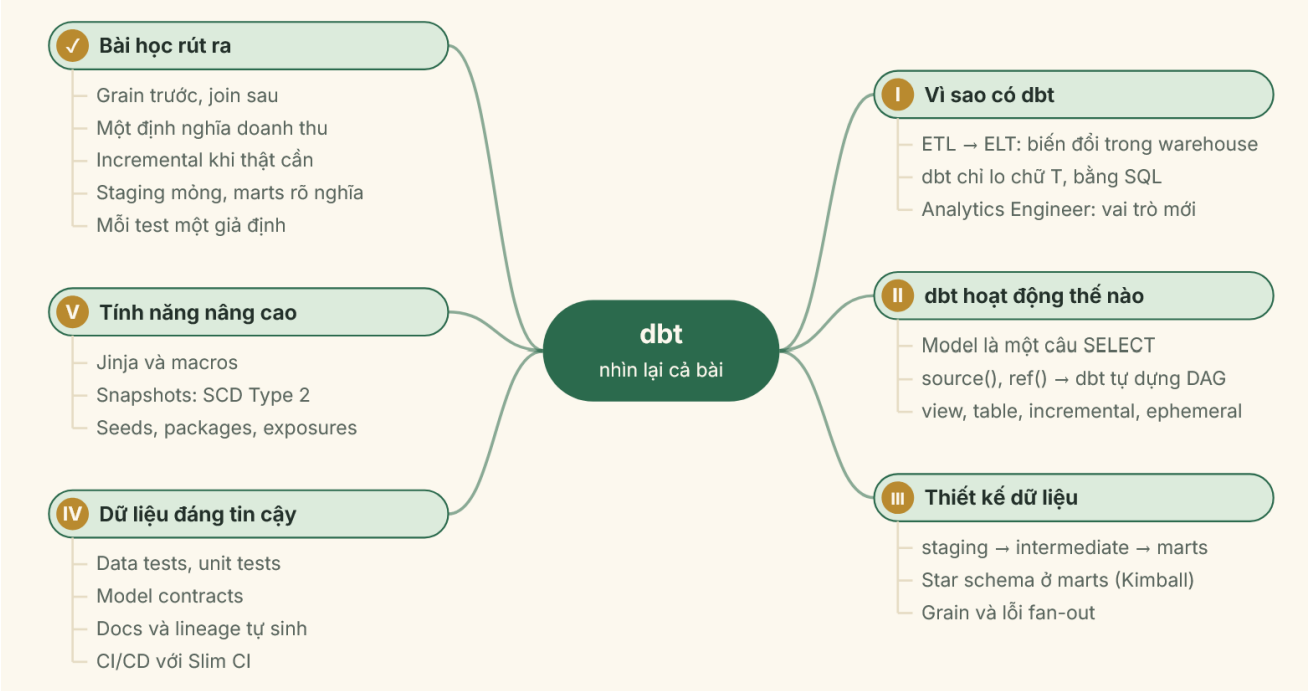
Để sử dụng, cần khai báo snapshot strategy và các cấu hình liên quan, sau đó chạy `dbt snapshot`. Lệnh này cần được thực thi định kỳ thông qua scheduler hoặc quy trình điều phối bên ngoài nếu muốn theo dõi lịch sử liên tục. Snapshot chỉ ghi nhận những thay đổi mà nó quan sát được tại các lần chạy, không thể khôi phục những trạng thái trung gian chưa từng được ghi nhận.

V.3. Seeds, packages và exposures

Ngoài model và test, dbt còn cung cấp một số thành phần hỗ trợ quản lý dữ liệu và mở rộng project:

- **Seeds:** các file CSV nhỏ được đặt trong thư mục `seeds/`. Có thể nạp vào warehouse bằng `dbt seed` và tham chiếu như model thông qua `ref()`. Seeds phù hợp với dữ liệu tĩnh, ít thay đổi như bảng ánh xạ hoặc danh sách mã phân loại.
- **Packages:** các gói mở rộng chứa macro, test và những thành phần có thể tái sử dụng. Packages được khai báo trong `packages.yml` và cài đặt bằng `dbt deps`. Ví dụ: `dbt_utils` và `dbt_expectations`.
- **Exposures:** khai báo các thành phần sử dụng dữ liệu đầu ra của dbt, chẳng hạn dashboard, báo cáo hoặc ứng dụng. Exposure giúp mở rộng lineage từ model đến nơi dữ liệu được sử dụng.

Kết luận



Hình 5: Sơ đồ tư duy tổng hợp các nội dung về dbt.

- **Bối cảnh ra đời:** ETL chuyển sang ELT; dbt tập trung vào bước Transform bằng SQL; Analytics Engineer kết nối kỹ thuật dữ liệu với nhu cầu phân tích.
- **Cách dbt hoạt động:** Model là đơn vị biến đổi; `source()` và `ref()` tạo DAG; materialization quyết định cách lưu kết quả dưới dạng view, table, incremental hoặc ephemeral.
- **Thiết kế dữ liệu:** staging → intermediate → marts; Star Schema theo Kimball; xác định grain để tránh lỗi fan-out.
- **Đảm bảo độ tin cậy:** Data tests, unit tests và model contracts; documentation và lineage; CI/CD với Slim CI.
- **Tính năng mở rộng:** Jinja và macros; snapshots để lưu lịch sử; seeds, packages và exposures.

Bài học rút ra

1. **Grain trước, join sau.** Trước mỗi phép join, cần xác định grain của các bảng đầu vào và grain của kết quả. Đây là bước quan trọng để tránh fan-out và sai lệch số liệu.
2. **Mỗi chỉ số nghiệp vụ cần một định nghĩa thống nhất.** Logic tính doanh thu nên được tổ chức tại một model có tài liệu và kiểm thử rõ ràng. Các model phía sau tham chiếu lại thông qua `ref()` thay vì tự tính lại theo những cách khác nhau.
3. **Chỉ dùng incremental khi thực sự cần thiết.** Bắt đầu với table để ưu tiên tính đơn giản và dễ kiểm soát. Chỉ chuyển sang incremental khi quy mô dữ liệu hoặc thời gian xử lý thực sự đòi hỏi, đồng thời phải hiểu rõ cơ chế cập nhật dữ liệu.
4. **Phân tầng để kiểm soát logic.** Staging tập trung làm sạch dữ liệu nguồn; intermediate xử lý các phép biến đổi và logic nghiệp vụ; marts cung cấp dữ liệu có cấu trúc rõ ràng cho người dùng phân tích.
5. **Mỗi test bảo vệ một giả định.** Test không chỉ để tăng số lượng kiểm tra, mà phải phản ánh những giả định quan trọng về khóa, quan hệ, giá trị và quy tắc nghiệp vụ mà các model đang dựa vào.